

Introducción a los Sistemas Operativos / Conceptos de Sistemas Operativos

Tema 2 – Anexo III Flujo de una System Call



- ✓ Versión: Septiembre 2026
- ✓ Palabras Claves: System Call, Biblioteca, Wrapper, Trap, Cambio de Modo, Tabla de System Calls, Bloqueante

Los temas vistos en estas diapositivas han sido mayormente extraídos de: Stallings, W. *Operating Systems: Internals and Design Principles*; Silberschatz, A., Galvin, P. y Gagne, G. *Operating System Concepts*.



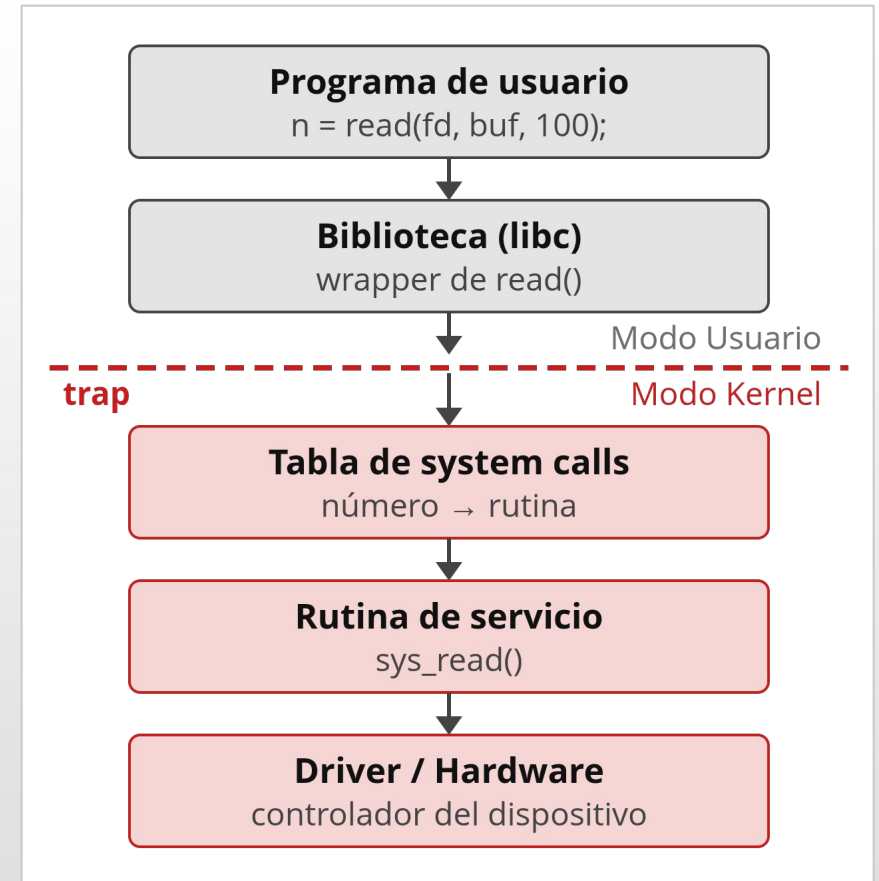
¿Qué es una System Call?

- ✓ Una **system call** es el punto de entrada controlado al Kernel para solicitar un servicio
 - ✓ Operaciones que el proceso no puede realizar en Modo Usuario: E/S, gestión de procesos, memoria
- ✓ Se analizan dos casos:
 - ✓ **No bloqueante**: getpid(), se atiende y retorna de inmediato
 - ✓ **Bloqueante**: read(), puede tener que esperar a un dispositivo
- ✓ El flujo se organiza en **4 fases** y **13 pasos**



Capas involucradas

- ✓ La **interfaz de system calls** es la frontera entre Modo Usuario y Modo Kernel
 - ✓ Por encima: el programa y la biblioteca
 - ✓ Por debajo: el Kernel y los drivers
- ✓ El programa depende de la **API** de la biblioteca, no de la system call concreta



La biblioteca como abstracción

- ✓ Los procesos no suelen invocar la system call directamente, sino a través de una **biblioteca** (API)
 - ◆ Por ejemplo, en UNIX/Linux se utiliza la libc (API POSIX); en Windows, la API Win32
- ✓ La función de biblioteca (**wrapper**) se encarga de:
 - ✓ Ubicar el número de system call y los parámetros
 - ✓ Ejecutar el trap y traducir el resultado
- ✓ Aporta **portabilidad**: el programa no depende de la arquitectura ni del Kernel concreto
 - ✓ No toda función de biblioteca invoca una system call (strlen no; printf sí, a través de write)



Visión general



Fase 1 – Invocación

El proceso prepara el pedido en Modo Usuario



Fase 1 – Invocación y parámetros

✓ **Paso 1 – Invocación:** el programa llama a la función de biblioteca como a cualquier otra función

✓ Ejemplos: `read(fd, buffer, n)` o `getpid()`

✓ **Paso 2 – Preparación:** el wrapper ubica el **número de system call** y los parámetros donde el Kernel los espera

✓ En registros, en un bloque de memoria cuya dirección va en un registro, o en la pila

◆ *Por ejemplo*, en los procesadores x86/x64 (Linux) el número va en RAX y los parámetros en RDI, RSI, RDX, R10, R8 y R9; en ARM64, el número en X8 y los parámetros en X0 a X5



Fase 1 – Instrucción de trap

✓ **Paso 3 – Trap:** el wrapper ejecuta una instrucción especial que genera una excepción intencional

✓ **No es una instrucción privilegiada:** justamente puede ejecutarse en Modo Usuario

✓ Es la única forma legítima de que un proceso ingrese al Kernel por su propia voluntad

◆ *Por ejemplo, en los procesadores x86/x64 se utiliza SYSCALL (en 32 bits, INT 0x80 o SYSENTER); en ARM, SVC*

✓ Hasta aquí todo ocurrió en **Modo Usuario**, dentro del proceso



Fase 2 – Trap

El hardware transfiere el control al Kernel



Fase 2 – Cambio de modo y Resguardo

✓ **Paso 4 – Cambio de modo:** el procesador pasa de Modo Usuario a Modo Kernel

◆ *Por ejemplo, en los procesadores x86/x64 el CPL pasa de ring 3 a ring 0; en ARM64, de EL0 a EL1*

✓ **Paso 5 – Resguardo mínimo:** el hardware guarda la dirección de retorno y el registro de estado

✓ Según la arquitectura, se guardan en registros o en la pila de kernel

◆ *Por ejemplo, en los procesadores x86/x64, SYSCALL copia RIP en RCX y RFLAGS en R11, sin cambiar de pila (lo hace luego el Kernel); INT 0x80 apila SS, ESP, EFLAGS, CS y EIP. En ARM64 se usan ELR_EL1 y SPSR_EL1*



Fase 2 – Punto de entrada al Kernel

✓ **Paso 6 – Salto al Kernel:** el PC se carga con un **punto de entrada fijo**, definido por el Kernel

- ✓ El proceso no elige a qué dirección salta: solo pide un servicio mediante un número
- ✓ Así se garantiza que en Modo Kernel solo se ejecute código del Kernel
 - ◆ *Por ejemplo*, en los procesadores x86/x64, SYSCALL toma la dirección del registro MSR LSTAR e INT 0x80 la obtiene de la IDT; en ARM64, de la tabla de vectores (VBAR_EL1)



Fase 3 – Atención en el Kernel

Despacho, validación y ejecución del servicio



Fase 3 – Entrada y despacho

- ✓ **Paso 7 – Entrada:** el código de entrada cambia a la pila de kernel (si el hardware no lo hizo) y resguarda los registros del proceso
 - ◆ *Por ejemplo, en Linux x86-64, la rutina entry_SYSCALL_64 arma la estructura pt_regs en la pila de kernel*
- ✓ **Paso 8 – Despacho:** el número de system call se valida y se usa como índice en la **tabla de system calls**, que contiene la dirección de cada rutina de servicio
 - ✓ Un número inválido se rechaza con un código de error
 - ◆ *Por ejemplo, en Linux x86-64 se accede a sys_call_table[RAX]: read es la 0 y getpid la 39*



Fase 3 – Validación de parámetros

✓ **Paso 9 – Validación:** el Kernel **no confía** en los parámetros que recibe del proceso

- ✓ Verifica que descriptores, tamaños y punteros sean válidos
- ✓ Comprueba que los punteros apunten al espacio del proceso y no al del Kernel
- ✓ Copia los datos entre ambos espacios con rutinas específicas

◆ *Por ejemplo, en Linux se utilizan `copy_from_user()` y `copy_to_user()`*

✓ Un parámetro inválido termina la system call con error, sin afectar al Kernel



Fase 3 – Caso no bloqueante: `getpid()`

- ✓ **Paso 10a – Ejecución del servicio:** la rutina lee el identificador del proceso desde su PCB
 - ✓ No requiere acceder a dispositivos ni esperar ningún evento
 - ✓ El valor se deja como resultado de la system call
- ✓ El proceso **nunca abandona el estado Running**
 - ✓ Solo hay dos cambios de modo: usuario → kernel → usuario
 - ✓ **No hay cambio de contexto**
- ✓ Se continúa directamente con la Fase 4



Fase 3 – Caso bloqueante: read()

- ✓ **Paso 10b – Ejecución del servicio:** la rutina verifica si los datos pedidos ya están disponibles
 - ✓ **Si lo están** (por ejemplo, en el buffer cache): se copian al buffer del proceso y se retorna sin bloquear
 - ✓ **Si no lo están:** el Kernel solicita la operación al driver del dispositivo y el proceso debe esperar
- ✓ Por eso read() es **potencialmente** bloqueante: depende de la disponibilidad de los datos
 - ✓ Por ejemplo, un read sobre el teclado espera a que el usuario escriba



Fase 3 – read(): bloqueo del proceso

- ✓ El proceso pasa de **Running** a **Waiting** y su PCB se encola en la cola del dispositivo
- ✓ Se invoca al Scheduler, que elige otro proceso, y se produce un **cambio de contexto**
 - ✓ El detalle de este mecanismo se ve en la transparencia **Tema 2 – Anexo II – Cambio de Contexto**
- ✓ Al finalizar la E/S, la **interrupción del dispositivo** hace que el proceso pase a **Ready**
- ✓ Cuando vuelve a ser elegido, continúa en Modo Kernel y copia los datos al buffer del proceso



read() bloqueante: línea de tiempo



Fase 4 – Retorno

El resultado vuelve al proceso



Fase 4 – Valor de retorno y regreso

✓ **Paso 11 – Valor de retorno:** la rutina deja el resultado en un registro acordado

✓ Un valor negativo indica error

◆ *Por ejemplo, en los procesadores x86/x64 (Linux) el resultado va en RAX; en ARM64, en X0*

✓ **Paso 12 – Retorno a Modo Usuario:** se restauran los registros y se ejecuta la instrucción de retorno

✓ El hardware restaura el PC y el registro de estado, y vuelve a Modo Usuario

◆ *Por ejemplo, en los procesadores x86/x64 se utiliza SYSRET (restaura RIP desde RCX y RFLAGS desde R11) o IRET; en ARM64, ERET*



Fase 4 – Regreso al programa

- ✓ **Paso 13 – Post-procesamiento en la biblioteca:** el wrapper interpreta el resultado del Kernel
 - ✓ Si hubo error, guarda el código en errno y devuelve -1
 - ✓ Si no, devuelve el resultado (bytes leídos, PID, etc.)
- ✓ El programa continúa en la instrucción siguiente a la llamada, como si fuera una función más
 - ✓ La complejidad del mecanismo queda **oculta** por la biblioteca



No bloqueante vs. bloqueante

getpid()

- ✓ Se resuelve con datos del PCB
- ✓ El proceso permanece en Running
- ✓ Dos cambios de modo, sin cambio de contexto
- ✓ Tiempo de respuesta muy corto

read()

- ✓ Puede depender de un dispositivo
- ✓ El proceso puede pasar a Waiting
- ✓ Si bloquea, implica cambios de contexto
- ✓ Tiempo de respuesta variable



Ideas clave

- ✓ La system call es el **único punto de entrada controlado** al Kernel
- ✓ La **biblioteca** oculta los detalles: número, registros y trap
- ✓ La instrucción de trap **no es privilegiada**; lo que ocurre después sí
- ✓ Toda system call implica **cambio de modo**; si bloquea, además provoca un cambio de contexto
- ✓ El Kernel **valida todo**: el número de system call y los parámetros

